

Digital Verification Course – Assignment 5

Full UVM Router Verification & Verbosity Demonstration Report

Author: Ziad Mostafa Abdelaziz

Automated Verification Suite | QuestaSim 2021.1 | UVM 1.1d / UVM 1.2

Contents

1	Introduction and Verification Objectives	2
1.1	DUT Protocol and Specification	2
1.2	Key Assignment Objectives	2
2	UVM Architecture and Component Details	2
2.1	SystemVerilog Interface (router_if.sv)	2
2.2	Configuration Object (router_config.sv)	2
2.3	Sequence Item (router_seq_item.sv)	2
2.4	Sequences & Corner-Case Stimulus (router_sequence.sv)	2
2.5	Driver (router_driver.sv)	3
2.6	Monitor (router_monitor.sv)	3
2.7	Agent (router_agent.sv)	3
2.8	Scoreboard (router_scoreboard.sv)	3
2.9	Coverage Collector (router_coverage.sv)	3
3	Automated Script (run.do)	3
4	UVM Verbosity Demonstration Analysis and Screenshots	4
4.1	Run 1: +UVM_VERBOSITY=UVM_LOW	4
4.2	Run 2: +UVM_VERBOSITY=UVM_MEDIUM	4
4.3	Run 3: +UVM_VERBOSITY=UVM_HIGH	5
4.4	Run 4: +UVM_VERBOSITY=UVM_FULL	5
4.5	Verbosity Message Count Comparison Summary	5
5	Functional Coverage Report (100% Achieved)	6
5.1	Functional Coverage Breakdown Table	6
5.2	Directed Stimulus for 100% Coverage & Corner-Case Analysis	6
6	Self-Verifying Scoreboard Verification Results	6
7	Complete SystemVerilog Source Code Listings	7
7.1	router.v (DUT)	7
7.2	router_if.sv (Interface)	7
7.3	router_config.sv (Configuration Object)	8
7.4	router_seq_item.sv (Sequence Item)	8
7.5	router_sequence.sv (Sequences)	9
7.6	router_sequencer.sv (Sequencer)	9
7.7	router_driver.sv (Driver)	10
7.8	router_monitor.sv (Monitor)	10
7.9	router_agent.sv (Agent)	11
7.10	router_scoreboard.sv (Scoreboard)	12
7.11	router_coverage.sv (Coverage Collector)	13
7.12	router_env.sv (Environment)	14
7.13	router_test.sv (Test)	14
7.14	top.sv (Top-Level Testbench Module)	15
8	Conclusion	16

Introduction and Verification Objectives

This report presents the complete Universal Verification Methodology (UVM) verification environment developed for the `router.v` Design Under Test (DUT).

DUT Protocol and Specification

The DUT is a 4-input, 2-output packet router (`router.v`) operating on a synchronous system clock (`clk`) and active-low reset (`rst_n`).

- **Input Ports:** Four 8-bit data inputs (`data_in0`, `data_in1`, `data_in2`, `data_in3`) accompanied by corresponding valid indicators (`valid_in0`, `valid_in1`, `valid_in2`, `valid_in3`).
- **Output Ports:** Two 8-bit data outputs (`data_out0`, `data_out1`) and valid indicators (`valid_out0`, `valid_out1`).
- **Routing Logic & Priority Protocol:**
 1. **Even Ports (Out 0):** Even-numbered inputs (Port 0 and Port 2) are routed to `data_out0`. When both `valid_in0` and `valid_in2` are active simultaneously, Port 0 takes strict priority over Port 2.
 2. **Odd Ports (Out 1):** Odd-numbered inputs (Port 1 and Port 3) are routed to `data_out1`. When both `valid_in1` and `valid_in3` are active simultaneously, Port 1 takes strict priority over Port 3.

Key Assignment Objectives

1. **Full UVM Testbench Hierarchy:** Construct a modular UVM environment adhering to standards (`uvm_agent`, `uvm_driver`, `uvm_monitor`, `uvm_sequencer`, `uvm_scoreboard`, `uvm_subscriber`, `uvm_env`, `uvm_test`).
2. **Dynamic Agent Mode Control:** Pass a custom configuration object (`router_config`) through `uvm_config_db` to dynamically set agent operation between `UVM_ACTIVE` (instantiating driver and sequencer) and `UVM_PASSIVE` (instantiating monitor only).
3. **Core Graded Requirement – UVM Verbosity Demonstration:** Include 4 explicit `uvm.info` messages (`UVM_LOW`, `UVM_MEDIUM`, `UVM_HIGH`, `UVM_FULL`) in the `build_phase` of *every* component to prove exact filtering across 4 simulation passes.
4. **100% Functional Coverage:** Implement functional covergroups covering input valids, data byte boundary values (`0x00`, `0xFF`), and priority cross-coverage.
5. **Self-Verifying Scoreboard:** Validate golden reference output matching with zero errors.

UVM Architecture and Component Details

SystemVerilog Interface (`router_if.sv`)

Defines signal connections between DUT and testbench components. Features dedicated clocking blocks (`drv_cb` and `mon_cb`) with 1ns input/output skew constraints to prevent setup/hold timing violations.

Configuration Object (`router_config.sv`)

Inherits from `uvm_object`. Encapsulates `is_active` (`uvm_active_passive_enum`), `num_ports`, `coverage_enable`, and `checks_enable`. It is registered with `uvm_config_db` by `router_test` and retrieved by `router_env` and `router_agent`.

Sequence Item (`router_seq_item.sv`)

Extends `uvm_sequence_item`. Defines randomized inputs (`data_in0..3`, `valid_in0..3`) and captured outputs (`data_out0..1`, `valid_out0..1`). Implements `uvm_field_int` macros for field automation.

Sequences & Corner-Case Stimulus (`router_sequence.sv`)

Provides directed scenarios followed by 100 constrained-random item iterations. It incorporates targeted corner-case transactions:

- **All-Inactive Ports:** Drives `valid_in0..3 = 0` to cover all inactive cross-coverage bins.
- **Minimum Byte Boundary (`0x00`):** Drives `data_in0..3 = 8'h00` across all ports to hit bins `zero = {0}`.
- **Maximum Byte Boundary (`0xFF`):** Drives `data_in0..3 = 8'hFF` across all ports to hit bins `max = {255}`.

Driver (router_driver.sv)

Retrieves items from sequencer via `seq_item_port` and drives them synchronously onto `vif.drv_cb` interface.

Monitor (router_monitor.sv)

Synchronously samples interface signals via `vif.mon_cb` and broadcasts captured transactions to scoreboard and coverage collector through a `uvm_analysis_port`.

Agent (router_agent.sv)

Retrieves `router_config` from `uvm_config_db` during `build_phase`. If `cfg.is_active == UVM_ACTIVE`, it instantiates driver and sequencer; if `UVM_PASSIVE`, it omits them. Connects driver to sequencer during `connect_phase`.

Scoreboard (router_scoreboard.sv)

Implements golden reference model comparing sampled DUT outputs against expected behavior:

$$ExpectedOut0 = valid_in0 ? data_in0 : data_in2 \quad (1)$$

$$ExpectedOut1 = valid_in1 ? data_in1 : data_in3 \quad (2)$$

Coverage Collector (router_coverage.sv)

Subscribes to monitor's analysis port and samples covergroup `cg_router` on each transaction.

Automated Script (run.do)

The automated Tcl script cleans the work library, compiles DUT and UVM files, optimizes design, and runs all 4 verbosity simulations cleanly.

Listing 1: Automated QuestaSim Script (run.do)

```

1 # =====
2 # run.do - Automated Compilation and 4 Verbosity Simulation Runs
3 # =====
4 if {[file exists work]} { vdel -all -lib work }
5 vlib work
6
7 vlog -sv -suppress 2286,2181 +incdir+C:/questasim64_2021.1/verilog_src/uvm-1.1d/src router.v router_pkg.sv router_if.sv top.sv
8 vopt top -o top_opt +acc -suppress 3009
9
10 vsim top_opt +UVM_VERBOSITY=UVM_LOW -l sim_uvm_low.log
11 onfinish final
12 run -all
13 quit -sim
14
15 vsim top_opt +UVM_VERBOSITY=UVM_MEDIUM -l sim_uvm_medium.log
16 onfinish final
17 run -all
18 quit -sim
19
20 vsim top_opt +UVM_VERBOSITY=UVM_HIGH -l sim_uvm_high.log
21 onfinish final
22 run -all
23 quit -sim
24
25 vsim top_opt +UVM_VERBOSITY=UVM_FULL -l sim_uvm_full.log
26 onfinish final
27 run -all
28 quit -sim

```

UVM Verbosity Demonstration Analysis and Screenshots

Every component (test, env, agent, driver, monitor, sequencer, scoreboard, coverage) includes 4 explicit uvm_info messages in build_phase. The figures and tables below verify message filtering.

Run 1: +UVM_VERBOSITY=UVM_LOW

Filters out all messages above UVM_LOW. Exactly 1 build_phase print per component is reported (Total UVM_INFO = 16). Coverage = 100.00%, Scoreboard = 236 PASS, 0 FAIL.

```
# UVM_INFO router_coverage.sv(87) @ 1215000: uvm_test_top.env.cov [router_coverage] FUNCTIONAL COVERAGE: 100.00%
# UVM_INFO router_scoreboard.sv(68) @ 1215000: uvm_test_top.env.sb [router_scoreboard] SCOREBOARD RESULT: PASS=236 FAIL=0
#
# --- UVM Report Summary ---
#
# ** Report counts by severity
# UVM_INFO : 16
# UVM_WARNING : 0
# UVM_ERROR : 0
# UVM_FATAL : 0
# ** Report counts by id
# [RNTST] 1
# [TEST_DONE] 1
# [router_agent] 2
# [router_coverage] 2
# [router_driver] 1
# [router_env] 1
# [router_monitor] 1
# [router_random_sequence] 2
# [router_scoreboard] 2
# [router_sequencer] 1
# [router_test] 2
# ** Note: $finish : C:/questasim64_2021.1/win64/./verilog_src/uvm-1.1d/src/base/uvm_root.svh(430)
# Time: 1215 ns Iteration: 54 Instance: /top
```

Run 2: +UVM_VERBOSITY=UVM_MEDIUM

Prints UVM_LOW and UVM_MEDIUM messages. Exactly 2 build_phase prints per component are recorded (Total UVM_INFO = 26).

```
# UVM_INFO router_coverage.sv(87) @ 1215000: uvm_test_top.env.cov [router_coverage] FUNCTIONAL COVERAGE: 100.00%
# UVM_INFO router_scoreboard.sv(68) @ 1215000: uvm_test_top.env.sb [router_scoreboard] SCOREBOARD RESULT: PASS=236 FAIL=0
#
# --- UVM Report Summary ---
#
# ** Report counts by severity
# UVM_INFO : 26
# UVM_WARNING : 0
# UVM_ERROR : 0
# UVM_FATAL : 0
# ** Report counts by id
# [Questa UVM] 2
# [RNTST] 1
# [TEST_DONE] 1
# [router_agent] 3
# [router_coverage] 3
# [router_driver] 2
# [router_env] 2
# [router_monitor] 2
# [router_random_sequence] 2
# [router_scoreboard] 3
# [router_sequencer] 2
# [router_test] 3
# ** Note: $finish : C:/questasim64_2021.1/win64/./verilog_src/uvm-1.1d/src/base/uvm_root.svh(430)
# Time: 1215 ns Iteration: 54 Instance: /top
```

Run 3: +UVM_VERBOSITY=UVM_HIGH

Prints *UVM_LOW*, *UVM_MEDIUM*, and *UVM_HIGH* messages. Exactly 3 *build_phase* prints per component are recorded. Scoreboard match prints during *run_phase* are included (Total *UVM_INFO* = 271).

```
# UVM_INFO router_coverage.sv(87) @ 1215000: uvm_test_top.env.cov [router_coverage] FUNCTIONAL COVERAGE: 100.00%
# UVM_INFO router_scoreboard.sv(68) @ 1215000: uvm_test_top.env.sb [router_scoreboard] SCOREBOARD RESULT: PASS=236 FAIL=0
#
# --- UVM Report Summary ---
#
# ** Report counts by severity
# UVM_INFO : 271
# UVM_WARNING : 0
# UVM_ERROR : 0
# UVM_FATAL : 0
# ** Report counts by id
# [Questa UVM] 3
# [RNTST] 1
# [TEST_DONE] 1
# [router_agent] 4
# [router_coverage] 4
# [router_driver] 3
# [router_env] 3
# [router_monitor] 3
# [router_random_sequence] 2
# [router_scoreboard] 240
# [router_sequencer] 3
# [router_test] 4
# ** Note: $finish : C:/questasim64_2021.1/win64/./verilog_src/uvm-1.1d/src/base/uvm_root.svh(430)
# Time: 1215 ns Iteration: 54 Instance: /top
```

Run 4: +UVM_VERBOSITY=UVM_FULL

Prints *ALL FOUR* verbosity levels per component. Monitor item prints are fully un-filtered (Total *UVM_INFO* = 411).

```
# UVM_INFO router_coverage.sv(87) @ 1215000: uvm_test_top.env.cov [router_coverage] FUNCTIONAL COVERAGE: 100.00%
# UVM_INFO router_scoreboard.sv(68) @ 1215000: uvm_test_top.env.sb [router_scoreboard] SCOREBOARD RESULT: PASS=236 FAIL=0
#
# --- UVM Report Summary ---
#
# ** Report counts by severity
# UVM_INFO : 411
# UVM_WARNING : 0
# UVM_ERROR : 0
# UVM_FATAL : 0
# ** Report counts by id
# [PHASESEQ] 13
# [Questa UVM] 3
# [RNTST] 1
# [TEST_DONE] 2
# [router_agent] 5
# [router_coverage] 5
# [router_driver] 4
# [router_env] 4
# [router_monitor] 122
# [router_random_sequence] 2
# [router_scoreboard] 241
# [router_sequencer] 4
# [router_test] 5
# ** Note: $finish : C:/questasim64_2021.1/win64/./verilog_src/uvm-1.1d/src/base/uvm_root.svh(430)
# Time: 1215 ns Iteration: 54 Instance: /top
```

Verbosity Message Count Comparison Summary

Component / ID	UVM_LOW	UVM_MEDIUM	UVM_HIGH	UVM_FULL
router.test	2	3	4	5
router.env	1	2	3	4
router.agent	2	3	4	5
router.driver	1	2	3	4
router.monitor	1	2	3	122
router.sequencer	1	2	3	4
router.scoreboard	2	3	240	241
router.coverage	2	3	4	5
Questa UVM / System	4	6	7	17
Total UVM_INFO Count	16	26	271	411

Functional Coverage Report (100% Achieved)

Functional Coverage Breakdown Table

Coverpoint / Cross Name	Target Bins	Hit Bins	Coverage %
cp_valid0 (inactive, active)	2	2	100.00%
cp_valid1 (inactive, active)	2	2	100.00%
cp_valid2 (inactive, active)	2	2	100.00%
cp_valid3 (inactive, active)	2	2	100.00%
cp_data0 (zero, low, high, max)	4	4	100.00%
cp_data1 (zero, low, high, max)	4	4	100.00%
cp_data2 (zero, low, high, max)	4	4	100.00%
cp_data3 (zero, low, high, max)	4	4	100.00%
cross_even_ports (valid0 × valid2)	4	4	100.00%
cross_odd_ports (valid1 × valid3)	4	4	100.00%
Overall Covergroup (cg_router)	32	32	100.00%

Directed Stimulus for 100% Coverage & Corner-Case Analysis

To bridge the coverage gap from 85.00% to ****100.00%****, specific directed sequence items were designed in `router_sequence.sv`:

- Boundary Byte 0x00 (Minimum Data):** Single-value bin `bins zero = {0}` requires exact byte value 8'h00. Generating a directed item with `data_in0..3 = 8'h00` guarantees hitting `bins zero` across all four data coverpoints.
- Boundary Byte 0xFF (Maximum Data):** Single-value bin `bins max = {255}` requires exact byte value 8'hFF. Generating a directed item with `data_in0..3 = 8'hFF` guarantees hitting `bins max` across all four data coverpoints.
- All-Inactive Port Cross Coverage:** Cross coverpoints `cross_even_ports` and `cross_odd_ports` require sampling both ports inactive (`valid_in = 0`). Driving an all-zero valid transaction satisfies all inactive cross combination bins.

Self-Verifying Scoreboard Verification Results

The scoreboard validates every sampled transaction during simulation:

```
# UVM_INFO router_coverage.sv(87) @ 1215000: uvm_test_top.env.cov [router_coverage] FUNCTIONAL COVERAGE: 100.00%
# UVM_INFO router_scoreboard.sv(68) @ 1215000: uvm_test_top.env.sb [router_scoreboard] SCOREBOARD RESULT: PASS=236 FAIL=0
# --- UVM Report Summary ---
# ** Report counts by severity
# UVM_INFO : 16
# UVM_WARNING : 0
# UVM_ERROR : 0
# UVM_FATAL : 0
```

Scoreboard Verification Metrics:

- **Total Routing Checks:** 236
- **Passed Checks:** 236 (100% pass rate)
- **Failed Checks:** 0
- **UVM Errors / Warnings:** 0

Complete SystemVerilog Source Code Listings

router.v (DUT)

```

1 module router (
2     input clk,
3     input rst_n,
4     input [7:0] data_in0,
5     input [7:0] data_in1,
6     input [7:0] data_in2,
7     input [7:0] data_in3,
8     input valid_in0,
9     input valid_in1,
10    input valid_in2,
11    input valid_in3,
12    output [7:0] data_out0,
13    output [7:0] data_out1,
14    output valid_out0,
15    output valid_out1
16 );
17 // Simple routing: even-numbered inputs go to out0, odd to out1
18 assign data_out0 = valid_in0 ? data_in0 : data_in2;
19 assign valid_out0 = valid_in0 | valid_in2;
20
21 assign data_out1 = valid_in1 ? data_in1 : data_in3;
22 assign valid_out1 = valid_in1 | valid_in3;
23
24 endmodule

```

router_if.sv (Interface)

```

1 //=====
2 // router_if.sv Interface for Router DUT
3 //=====
4 `timescale 1ns/1ps
5
6 interface router_if (
7     input logic clk,
8     input logic rst_n
9 );
10 // Data inputs
11 logic [7:0] data_in0;
12 logic [7:0] data_in1;
13 logic [7:0] data_in2;
14 logic [7:0] data_in3;
15
16 // Valid inputs
17 logic valid_in0;
18 logic valid_in1;
19 logic valid_in2;
20 logic valid_in3;
21
22 // Data outputs
23 logic [7:0] data_out0;
24 logic [7:0] data_out1;
25
26 // Valid outputs
27 logic valid_out0;
28 logic valid_out1;
29
30 // Driver Clocking Block
31 clocking drv_cb @(posedge clk);
32     default input #1ns output #1ns;
33     output data_in0, data_in1, data_in2, data_in3;
34     output valid_in0, valid_in1, valid_in2, valid_in3;
35     input data_out0, data_out1;
36     input valid_out0, valid_out1;
37 endclocking
38
39 // Monitor Clocking Block
40 clocking mon_cb @(posedge clk);
41     default input #1ns output #1ns;
42     input data_in0, data_in1, data_in2, data_in3;
43     input valid_in0, valid_in1, valid_in2, valid_in3;
44     input data_out0, data_out1;
45     input valid_out0, valid_out1;
46 endclocking
47
48 // Modports
49 modport DRIVER (clocking drv_cb, input clk, input rst_n);
50 modport MONITOR (clocking mon_cb, input clk, input rst_n);
51 modport DUT (
52     input clk, rst_n,
53     input data_in0, data_in1, data_in2, data_in3,
54     input valid_in0, valid_in1, valid_in2, valid_in3,
55     output data_out0, data_out1,
56     output valid_out0, valid_out1
57 );
58

```

```
59 endinterface : router_if
```

router_config.sv (Configuration Object)

```
1 //=====
2 // router_config.sv  Environment & Agent Configuration Object
3 //=====
4 `timescale 1ns/1ps
5
6 class router_config extends uvm_object;
7
8     `uvm_object_utils(router_config)
9
10    // Agent active/passive control via uvm_active_passive_enum
11    uvm_active_passive_enum is_active = UVM_ACTIVE;
12
13    // Environment settings
14    int num_ports = 4;
15    bit coverage_enable = 1'b1;
16    bit checks_enable = 1'b1;
17
18    function new(string name = "router_config");
19        super.new(name);
20    endfunction
21
22 endclass : router_config
```

router_seq_item.sv (Sequence Item)

```
1 //=====
2 // router_seq_item.sv  Sequence Item modeling Router Transactions
3 //=====
4 `timescale 1ns/1ps
5
6 class router_seq_item extends uvm_sequence_item;
7
8    // Random stimulus fields (Inputs to Router)
9    rand bit [7:0] data_in0;
10   rand bit [7:0] data_in1;
11   rand bit [7:0] data_in2;
12   rand bit [7:0] data_in3;
13
14   rand bit valid_in0;
15   rand bit valid_in1;
16   rand bit valid_in2;
17   rand bit valid_in3;
18
19   // Response / Sampled fields (Outputs from Router)
20   bit [7:0] data_out0;
21   bit [7:0] data_out1;
22   bit valid_out0;
23   bit valid_out1;
24
25   // Constraints for balanced stimulus generation
26   constraint c_valid_dist {
27       valid_in0 dist {1 := 70, 0 := 30};
28       valid_in1 dist {1 := 70, 0 := 30};
29       valid_in2 dist {1 := 70, 0 := 30};
30       valid_in3 dist {1 := 70, 0 := 30};
31   }
32
33   // UVM Field Macros
34   `uvm_object_utils_begin(router_seq_item)
35   `uvm_field_int(data_in0, UVM_ALL_ON | UVM_HEX)
36   `uvm_field_int(data_in1, UVM_ALL_ON | UVM_HEX)
37   `uvm_field_int(data_in2, UVM_ALL_ON | UVM_HEX)
38   `uvm_field_int(data_in3, UVM_ALL_ON | UVM_HEX)
39   `uvm_field_int(valid_in0, UVM_ALL_ON | UVM_BIN)
40   `uvm_field_int(valid_in1, UVM_ALL_ON | UVM_BIN)
41   `uvm_field_int(valid_in2, UVM_ALL_ON | UVM_BIN)
42   `uvm_field_int(valid_in3, UVM_ALL_ON | UVM_BIN)
43   `uvm_field_int(data_out0, UVM_ALL_ON | UVM_HEX)
44   `uvm_field_int(data_out1, UVM_ALL_ON | UVM_HEX)
45   `uvm_field_int(valid_out0, UVM_ALL_ON | UVM_BIN)
46   `uvm_field_int(valid_out1, UVM_ALL_ON | UVM_BIN)
47   `uvm_object_utils_end
48
49   function new(string name = "router_seq_item");
50       super.new(name);
51   endfunction
52
53   virtual function string convert2string();
54       return $sprintf("IN:[v0=%0b d0=0x%0h | v1=%0b d1=0x%0h | v2=%0b d2=0x%0h | v3=%0b d3=0x%0h] OUT:[v0=%0b d0=0x%0h | v1=%0b d1=0x%0h]",
55           valid_in0, data_in0, valid_in1, data_in1,
56           valid_in2, data_in2, valid_in3, data_in3,
57           valid_out0, data_out0, valid_out1, data_out1);
```



```

58     endfunction
59
60 endclass : router_seq_item

```

router_sequence.sv (Sequences)

```

1  //=====
2  // router_sequence.sv Router Stimulus Sequences
3  //=====
4  `timescale 1ns/1ps
5
6  class router_base_sequence extends uvm_sequence #(router_seq_item);
7      `uvm_object_utils(router_base_sequence)
8
9      function new(string name = "router_base_sequence");
10         super.new(name);
11     endfunction
12
13     virtual task body();
14         `uvm_info(get_type_name(), "Executing router_base_sequence body", UVM_HIGH)
15     endtask
16 endclass : router_base_sequence
17
18
19 class router_random_sequence extends router_base_sequence;
20     `uvm_object_utils(router_random_sequence)
21
22     int unsigned num_items = 100;
23
24     function new(string name = "router_random_sequence");
25         super.new(name);
26     endfunction
27
28     virtual task body();
29         `uvm_info(get_type_name(), $sformatf("Starting router_random_sequence with %0d items", num_items), UVM_LOW)
30
31         // 1. Directed scenarios to guarantee 100% functional coverage
32         // Inactive combinations for cross coverage
33         `uvm_do_with(req, { valid_in0 == 0; valid_in1 == 0; valid_in2 == 0; valid_in3 == 0; })
34
35         // Port 0 only active -> out0
36         `uvm_do_with(req, { valid_in0 == 1; valid_in1 == 0; valid_in2 == 0; valid_in3 == 0; data_in0 == 8'hA1; })
37         // Port 2 only active -> out0
38         `uvm_do_with(req, { valid_in0 == 0; valid_in1 == 0; valid_in2 == 1; valid_in3 == 0; data_in2 == 8'hB2; })
39         // Port 1 only active -> out1
40         `uvm_do_with(req, { valid_in0 == 0; valid_in1 == 1; valid_in2 == 0; valid_in3 == 0; data_in1 == 8'hC3; })
41         // Port 3 only active -> out1
42         `uvm_do_with(req, { valid_in0 == 0; valid_in1 == 0; valid_in2 == 0; valid_in3 == 1; data_in3 == 8'hD4; })
43
44         // Port 0 & Port 2 active simultaneously (Priority test: Port 0 must win on out0)
45         `uvm_do_with(req, { valid_in0 == 1; valid_in1 == 0; valid_in2 == 1; valid_in3 == 0; data_in0 == 8'hE5; data_in2 == 8'hE6; })
46         // Port 1 & Port 3 active simultaneously (Priority test: Port 1 must win on out1)
47         `uvm_do_with(req, { valid_in0 == 0; valid_in1 == 1; valid_in2 == 0; valid_in3 == 1; data_in1 == 8'hF7; data_in3 == 8'hF8; })
48
49         // Corner case values: Zero (0x00) on all ports
50         `uvm_do_with(req, { valid_in0 == 1; valid_in1 == 1; valid_in2 == 1; valid_in3 == 1; data_in0 == 8'h00; data_in1 == 8'h00;
51             data_in2 == 8'h00; data_in3 == 8'h00; })
52
53         // Corner case values: Max (0xFF) on all ports
54         `uvm_do_with(req, { valid_in0 == 1; valid_in1 == 1; valid_in2 == 1; valid_in3 == 1; data_in0 == 8'hFF; data_in1 == 8'hFF;
55             data_in2 == 8'hFF; data_in3 == 8'hFF; })
56
57         // 2. Bulk randomized transactions
58         repeat (num_items) begin
59             req = router_seq_item::type_id::create("req");
60             start_item(req);
61             if (!req.randomize()) begin
62                 `uvm_error(get_type_name(), "Randomization failed for router_seq_item!")
63             end
64             finish_item(req);
65         end
66
67         `uvm_info(get_type_name(), "Finished router_random_sequence execution", UVM_LOW)
68     endtask
69 endclass : router_random_sequence

```

router_sequencer.sv (Sequencer)

```

1  //=====
2  // router_sequencer.sv Router Sequencer Component
3  //=====
4  `timescale 1ns/1ps
5
6  class router_sequencer extends uvm_sequencer #(router_seq_item);
7
8      `uvm_component_utils(router_sequencer)
9

```

```

10 function new(string name = "router_sequencer", uvm_component parent = null);
11     super.new(name, parent);
12 endfunction
13
14 virtual function void build_phase(uvm_phase phase);
15     super.build_phase(phase);
16     // Verbosity Demonstration Prints
17     'uvm_info(get_type_name(), "Build Phase - UVM_LOW verbosity message", UVM_LOW)
18     'uvm_info(get_type_name(), "Build Phase - UVM_MEDIUM verbosity message", UVM_MEDIUM)
19     'uvm_info(get_type_name(), "Build Phase - UVM_HIGH verbosity message", UVM_HIGH)
20     'uvm_info(get_type_name(), "Build Phase - UVM_FULL verbosity message", UVM_FULL)
21 endfunction
22
23 endclass : router_sequencer

```

router_driver.sv (Driver)

```

1 //=====
2 // router_driver.sv Router Driver Component
3 //=====
4 `timescale 1ns/1ps
5
6 class router_driver extends uvm_driver #(router_seq_item);
7
8     'uvm_component_utils(router_driver)
9
10    virtual router_if vif;
11
12    function new(string name = "router_driver", uvm_component parent = null);
13        super.new(name, parent);
14    endfunction
15
16    virtual function void build_phase(uvm_phase phase);
17        super.build_phase(phase);
18        // Verbosity Demonstration Prints
19        'uvm_info(get_type_name(), "Build Phase - UVM_LOW verbosity message", UVM_LOW)
20        'uvm_info(get_type_name(), "Build Phase - UVM_MEDIUM verbosity message", UVM_MEDIUM)
21        'uvm_info(get_type_name(), "Build Phase - UVM_HIGH verbosity message", UVM_HIGH)
22        'uvm_info(get_type_name(), "Build Phase - UVM_FULL verbosity message", UVM_FULL)
23
24        // Retrieve Virtual Interface from Config DB
25        if (!uvm_config_db#(virtual router_if)::get(this, "", "vif", vif)) begin
26            'uvm_fatal(get_type_name(), "Virtual interface router_if not found in uvm_config_db!")
27        end
28    endfunction
29
30    virtual task run_phase(uvm_phase phase);
31        reset_signals();
32        wait (vif.rst_n == 1'b1);
33        @(vif.drv_cb);
34
35        forever begin
36            seq_item_port.get_next_item(req);
37            drive_item(req);
38            seq_item_port.item_done();
39        end
40    endtask
41
42    virtual task reset_signals();
43        vif.drv_cb.data_in0 <= 8'h00;
44        vif.drv_cb.data_in1 <= 8'h00;
45        vif.drv_cb.data_in2 <= 8'h00;
46        vif.drv_cb.data_in3 <= 8'h00;
47        vif.drv_cb.valid_in0 <= 1'b0;
48        vif.drv_cb.valid_in1 <= 1'b0;
49        vif.drv_cb.valid_in2 <= 1'b0;
50        vif.drv_cb.valid_in3 <= 1'b0;
51    endtask
52
53    virtual task drive_item(router_seq_item item);
54        vif.drv_cb.data_in0 <= item.data_in0;
55        vif.drv_cb.data_in1 <= item.data_in1;
56        vif.drv_cb.data_in2 <= item.data_in2;
57        vif.drv_cb.data_in3 <= item.data_in3;
58        vif.drv_cb.valid_in0 <= item.valid_in0;
59        vif.drv_cb.valid_in1 <= item.valid_in1;
60        vif.drv_cb.valid_in2 <= item.valid_in2;
61        vif.drv_cb.valid_in3 <= item.valid_in3;
62
63        @(vif.drv_cb);
64    endtask
65
66 endclass : router_driver

```

router_monitor.sv (Monitor)

```

1 //=====

```

```

2 // router_monitor.sv Router Monitor Component
3 //=====
4 `timescale 1ns/1ps
5
6 class router_monitor extends uvm_monitor;
7
8     'uvm_component_utils(router_monitor)
9
10    virtual router_if vif;
11    uvm_analysis_port #(router_seq_item) item_collected_port;
12
13    function new(string name = "router_monitor", uvm_component parent = null);
14        super.new(name, parent);
15    endfunction
16
17    virtual function void build_phase(uvm_phase phase);
18        super.build_phase(phase);
19        // Verbosity Demonstration Prints
20        'uvm_info(get_type_name(), "Build Phase - UVM_LOW verbosity message", UVM_LOW)
21        'uvm_info(get_type_name(), "Build Phase - UVM_MEDIUM verbosity message", UVM_MEDIUM)
22        'uvm_info(get_type_name(), "Build Phase - UVM_HIGH verbosity message", UVM_HIGH)
23        'uvm_info(get_type_name(), "Build Phase - UVM_FULL verbosity message", UVM_FULL)
24
25        // Retrieve Virtual Interface from Config DB
26        if (!uvm_config_db#(virtual router_if)::get(this, "", "vif", vif)) begin
27            'uvm_fatal(get_type_name(), "Virtual interface router_if not found in uvm_config_db!")
28        end
29
30        item_collected_port = new("item_collected_port", this);
31    endfunction
32
33    virtual task run_phase(uvm_phase phase);
34        wait (vif.rst_n === 1'b1);
35        @(vif.mon_cb);
36
37        forever begin
38            router_seq_item item = router_seq_item::type_id::create("item");
39
40            @(vif.mon_cb);
41            // Sample input port signals
42            item.data_in0 = vif.mon_cb.data_in0;
43            item.data_in1 = vif.mon_cb.data_in1;
44            item.data_in2 = vif.mon_cb.data_in2;
45            item.data_in3 = vif.mon_cb.data_in3;
46
47            item.valid_in0 = vif.mon_cb.valid_in0;
48            item.valid_in1 = vif.mon_cb.valid_in1;
49            item.valid_in2 = vif.mon_cb.valid_in2;
50            item.valid_in3 = vif.mon_cb.valid_in3;
51
52            // Sample output port signals
53            item.data_out0 = vif.mon_cb.data_out0;
54            item.data_out1 = vif.mon_cb.data_out1;
55            item.valid_out0 = vif.mon_cb.valid_out0;
56            item.valid_out1 = vif.mon_cb.valid_out1;
57
58            'uvm_info(get_type_name(), $sformatf("Sampled Item: %s", item.convert2string()), UVM_FULL)
59            item_collected_port.write(item);
60        end
61    endtask
62
63 endclass : router_monitor

```

router_agent.sv (Agent)

```

1 //=====
2 // router_agent.sv Router Agent Component (Config DB Active/Passive Control)
3 //=====
4 `timescale 1ns/1ps
5
6 class router_agent extends uvm_agent;
7
8     'uvm_component_utils(router_agent)
9
10    router_config cfg;
11    router_driver driver;
12    router_monitor monitor;
13    router_sequencer sequencer;
14
15    function new(string name = "router_agent", uvm_component parent = null);
16        super.new(name, parent);
17    endfunction
18
19    virtual function void build_phase(uvm_phase phase);
20        super.build_phase(phase);
21        // Verbosity Demonstration Prints
22        'uvm_info(get_type_name(), "Build Phase - UVM_LOW verbosity message", UVM_LOW)
23        'uvm_info(get_type_name(), "Build Phase - UVM_MEDIUM verbosity message", UVM_MEDIUM)
24        'uvm_info(get_type_name(), "Build Phase - UVM_HIGH verbosity message", UVM_HIGH)
25        'uvm_info(get_type_name(), "Build Phase - UVM_FULL verbosity message", UVM_FULL)

```

```

26
27 // Retrieve Agent/Env Config Object from Config DB
28 if (!uvm_config_db#(router_config)::get(this, "", "cfg", cfg)) begin
29     'uvm_info(get_type_name(), "No router_config found in uvm_config_db, creating default config", UVM_MEDIUM)
30     cfg = router_config::type_id::create("cfg");
31 end
32
33 // Dynamically set agent is_active mode from configuration object rather than hardcoding
34 is_active = cfg.is_active;
35 'uvm_info(get_type_name(), $sformatf("Agent configured mode: is_active = %s", is_active.name()), UVM_LOW)
36
37 // Create driver & sequencer ONLY if active
38 if (is_active == UVM_ACTIVE) begin
39     driver = router_driver::type_id::create("driver", this);
40     sequencer = router_sequencer::type_id::create("sequencer", this);
41 end
42
43 // Always create monitor
44 monitor = router_monitor::type_id::create("monitor", this);
45 endfunction
46
47 virtual function void connect_phase(uvm_phase phase);
48     super.connect_phase(phase);
49     if (is_active == UVM_ACTIVE) begin
50         driver.seq_item_port.connect(sequencer.seq_item_export);
51     end
52 endfunction
53
54 endclass : router_agent

```

router_scoreboard.sv (Scoreboard)

```

1 //=====
2 // router_scoreboard.sv Self-Verifying Scoreboard
3 //=====
4 `timescale 1ns/1ps
5
6 class router_scoreboard extends uvm_scoreboard;
7
8     'uvm_component_utils(router_scoreboard)
9
10     uvm_analysis_imp #(router_seq_item, router_scoreboard) item_imp;
11
12     int unsigned pass_count = 0;
13     int unsigned fail_count = 0;
14
15     function new(string name = "router_scoreboard", uvm_component parent = null);
16         super.new(name, parent);
17     endfunction
18
19     virtual function void build_phase(uvm_phase phase);
20         super.build_phase(phase);
21         // Verbosity Demonstration Prints
22         'uvm_info(get_type_name(), "Build Phase - UVM_LOW verbosity message", UVM_LOW)
23         'uvm_info(get_type_name(), "Build Phase - UVM_MEDIUM verbosity message", UVM_MEDIUM)
24         'uvm_info(get_type_name(), "Build Phase - UVM_HIGH verbosity message", UVM_HIGH)
25         'uvm_info(get_type_name(), "Build Phase - UVM_FULL verbosity message", UVM_FULL)
26
27         item_imp = new("item_imp", this);
28     endfunction
29
30     virtual function void write(router_seq_item item);
31         bit exp_valid_out0;
32         bit [7:0] exp_data_out0;
33         bit exp_valid_out1;
34         bit [7:0] exp_data_out1;
35
36         // Golden Reference Routing Logic:
37         // Even-numbered inputs (0, 2) route to out0; Priority: in0 > in2
38         exp_valid_out0 = item.valid_in0 | item.valid_in2;
39         exp_data_out0 = item.valid_in0 ? item.data_in0 : item.data_in2;
40
41         // Odd-numbered inputs (1, 3) route to out1; Priority: in1 > in3
42         exp_valid_out1 = item.valid_in1 | item.valid_in3;
43         exp_data_out1 = item.valid_in1 ? item.data_in1 : item.data_in3;
44
45         // Compare Output 0
46         if (item.valid_out0 != exp_valid_out0 || (exp_valid_out0 && item.data_out0 != exp_data_out0)) begin
47             fail_count++;
48             'uvm_error(get_type_name(), $sformatf("MISMATCH Out0! Act:[v=%0b d=0x%0h] Exp:[v=%0b d=0x%0h] Item: %s",
49                 item.valid_out0, item.data_out0, exp_valid_out0, exp_data_out0, item.convert2string()))
50         end else begin
51             pass_count++;
52             'uvm_info(get_type_name(), $sformatf("MATCH Out0! Act:[v=%0b d=0x%0h]", item.valid_out0, item.data_out0), UVM_HIGH)
53         end
54
55         // Compare Output 1
56         if (item.valid_out1 != exp_valid_out1 || (exp_valid_out1 && item.data_out1 != exp_data_out1)) begin
57             fail_count++;
58             'uvm_error(get_type_name(), $sformatf("MISMATCH Out1! Act:[v=%0b d=0x%0h] Exp:[v=%0b d=0x%0h] Item: %s",

```

```

59         item.valid_out1, item.data_out1, exp_valid_out1, exp_data_out1, item.convert2string()))
60     end else begin
61         pass_count++;
62         'uvm_info(get_type_name(), $sformatf("MATCH Out1! Act:[v=%0b d=0x%0h]", item.valid_out1, item.data_out1), UVM_HIGH)
63     end
64 endfunction
65
66 virtual function void report_phase(uvm_phase phase);
67     super.report_phase(phase);
68     'uvm_info(get_type_name(), $sformatf("SCOREBOARD RESULT: PASS=%0d FAIL=%0d", pass_count, fail_count), UVM_LOW)
69 endfunction
70
71 endclass : router_scoreboard

```

router_coverage.sv (Coverage Collector)

```

1  //=====
2  // router_coverage.sv Functional Coverage Collector
3  //=====
4  `timescale 1ns/1ps
5
6  class router_coverage extends uvm_subscriber #(router_seq_item);
7
8      'uvm_component_utils(router_coverage)
9
10     router_seq_item sampled_item;
11
12     covergroup cg_router;
13         option.per_instance = 1;
14         option.name = "cg_router";
15
16         cp_valid0: coverpoint sampled_item.valid_in0 {
17             bins inactive = {0};
18             bins active = {1};
19         }
20         cp_valid1: coverpoint sampled_item.valid_in1 {
21             bins inactive = {0};
22             bins active = {1};
23         }
24         cp_valid2: coverpoint sampled_item.valid_in2 {
25             bins inactive = {0};
26             bins active = {1};
27         }
28         cp_valid3: coverpoint sampled_item.valid_in3 {
29             bins inactive = {0};
30             bins active = {1};
31         }
32
33         cp_data0: coverpoint sampled_item.data_in0 {
34             bins zero = {0};
35             bins low = {[1:127]};
36             bins high = {[128:254]};
37             bins max = {255};
38         }
39         cp_data1: coverpoint sampled_item.data_in1 {
40             bins zero = {0};
41             bins low = {[1:127]};
42             bins high = {[128:254]};
43             bins max = {255};
44         }
45         cp_data2: coverpoint sampled_item.data_in2 {
46             bins zero = {0};
47             bins low = {[1:127]};
48             bins high = {[128:254]};
49             bins max = {255};
50         }
51         cp_data3: coverpoint sampled_item.data_in3 {
52             bins zero = {0};
53             bins low = {[1:127]};
54             bins high = {[128:254]};
55             bins max = {255};
56         }
57
58         // Cross coverage to verify even-numbered port priority (Out 0)
59         cross_even_ports: cross cp_valid0, cp_valid2;
60
61         // Cross coverage to verify odd-numbered port priority (Out 1)
62         cross_odd_ports: cross cp_valid1, cp_valid3;
63
64     endgroup
65
66     function new(string name = "router_coverage", uvm_component parent = null);
67         super.new(name, parent);
68         cg_router = new();
69     endfunction
70
71     virtual function void build_phase(uvm_phase phase);
72         super.build_phase(phase);
73         // Verbosity Demonstration Prints
74         'uvm_info(get_type_name(), "Build Phase - UVM_LOW verbosity message", UVM_LOW)

```

```

75     'uvm_info(get_type_name(), "Build Phase - UVM_MEDIUM verbosity message", UVM_MEDIUM)
76     'uvm_info(get_type_name(), "Build Phase - UVM_HIGH verbosity message", UVM_HIGH)
77     'uvm_info(get_type_name(), "Build Phase - UVM_FULL verbosity message", UVM_FULL)
78 endfunction
79
80 virtual function void write(router_seq_item t);
81     sampled_item = t;
82     cg_router.sample();
83 endfunction
84
85 virtual function void report_phase(uvm_phase phase);
86     super.report_phase(phase);
87     'uvm_info(get_type_name(), $sformatf("FUNCTIONAL COVERAGE: %0.2f%%", cg_router.get_coverage()), UVM_LOW)
88 endfunction
89
90 endclass : router_coverage

```

router_env.sv (Environment)

```

1 //=====
2 // router_env.sv Top-Level UVM Environment
3 //=====
4 `timescale 1ns/1ps
5
6 class router_env extends uvm_env;
7
8     'uvm_component_utils(router_env)
9
10    router_config cfg;
11    router_agent agent;
12    router_scoreboard sb;
13    router_coverage cov;
14
15    function new(string name = "router_env", uvm_component parent = null);
16        super.new(name, parent);
17    endfunction
18
19    virtual function void build_phase(uvm_phase phase);
20        super.build_phase(phase);
21        // Verbosity Demonstration Prints
22        'uvm_info(get_type_name(), "Build Phase - UVM_LOW verbosity message", UVM_LOW)
23        'uvm_info(get_type_name(), "Build Phase - UVM_MEDIUM verbosity message", UVM_MEDIUM)
24        'uvm_info(get_type_name(), "Build Phase - UVM_HIGH verbosity message", UVM_HIGH)
25        'uvm_info(get_type_name(), "Build Phase - UVM_FULL verbosity message", UVM_FULL)
26
27        // Retrieve or Create Config Object
28        if (!uvm_config_db#(router_config)::get(this, "", "cfg", cfg)) begin
29            'uvm_info(get_type_name(), "Creating default router_config inside router_env", UVM_MEDIUM)
30            cfg = router_config::type_id::create("cfg");
31        end
32
33        // Pass config object down to agent
34        uvm_config_db#(router_config)::set(this, "agent", "cfg", cfg);
35
36        // Instantiate sub-components
37        agent = router_agent::type_id::create("agent", this);
38
39        if (cfg.checks_enable) begin
40            sb = router_scoreboard::type_id::create("sb", this);
41        end
42        if (cfg.coverage_enable) begin
43            cov = router_coverage::type_id::create("cov", this);
44        end
45    endfunction
46
47    virtual function void connect_phase(uvm_phase phase);
48        super.connect_phase(phase);
49        if (cfg.checks_enable && sb != null) begin
50            agent.monitor.item_collected_port.connect(sb.item_imp);
51        end
52        if (cfg.coverage_enable && cov != null) begin
53            agent.monitor.item_collected_port.connect(cov.analysis_export);
54        end
55    endfunction
56
57 endclass : router_env

```

router_test.sv (Test)

```

1 //=====
2 // router_test.sv Base UVM Test
3 //=====
4 `timescale 1ns/1ps
5
6 class router_test extends uvm_test;
7
8     'uvm_component_utils(router_test)

```

```

9
10 router_env env;
11 router_config cfg;
12
13 function new(string name = "router_test", uvm_component parent = null);
14     super.new(name, parent);
15 endfunction
16
17 virtual function void build_phase(uvm_phase phase);
18     super.build_phase(phase);
19     // Verbosity Demonstration Prints
20     'uvm_info(get_type_name(), "Build Phase - UVM_LOW verbosity message", UVM_LOW)
21     'uvm_info(get_type_name(), "Build Phase - UVM_MEDIUM verbosity message", UVM_MEDIUM)
22     'uvm_info(get_type_name(), "Build Phase - UVM_HIGH verbosity message", UVM_HIGH)
23     'uvm_info(get_type_name(), "Build Phase - UVM_FULL verbosity message", UVM_FULL)
24
25     // Instantiate and configure the router_config object
26     cfg = router_config::type_id::create("cfg");
27     cfg.is_active = UVM_ACTIVE;
28     cfg.coverage_enable = 1'b1;
29     cfg.checks_enable = 1'b1;
30
31     // Set router_config object in uvm_config_db for env and sub-components
32     uvm_config_db#(router_config)::set(this, "env*", "cfg", cfg);
33
34     // Instantiate top-level environment
35     env = router_env::type_id::create("env", this);
36 endfunction
37
38 virtual task run_phase(uvm_phase phase);
39     router_random_sequence seq;
40     phase.raise_objection(this, "Starting router_test execution");
41
42     'uvm_info(get_type_name(), "Executing router_test run_phase", UVM_LOW)
43     seq = router_random_sequence::type_id::create("seq");
44     seq.start(env.agent.sequencer);
45
46     #100ns;
47     phase.drop_objection(this, "Finished router_test execution");
48 endtask
49
50 endclass : router_test

```

top.sv (Top-Level Testbench Module)

```

1 //=====
2 // top.sv Top-Level Testbench Module
3 //=====
4 `timescale 1ns/1ps
5
6 import uvm_pkg::*;
7 `include "uvm_macros.svh"
8 import router_pkg::*;
9
10 module top;
11
12     // Clock and Reset Signals
13     logic clk;
14     logic rst_n;
15
16     // Clock Generation (100 MHz clock)
17     initial begin
18         clk = 0;
19         forever #5ns clk = ~clk;
20     end
21
22     // Reset Generation
23     initial begin
24         rst_n = 0;
25         #20ns rst_n = 1;
26     end
27
28     // Interface Instantiation
29     router_if intf (
30         .clk (clk),
31         .rst_n (rst_n)
32     );
33
34     // DUT Instantiation
35     router dut (
36         .clk (intf.clk),
37         .rst_n (intf.rst_n),
38         .data_in0 (intf.data_in0),
39         .data_in1 (intf.data_in1),
40         .data_in2 (intf.data_in2),
41         .data_in3 (intf.data_in3),
42         .valid_in0 (intf.valid_in0),
43         .valid_in1 (intf.valid_in1),
44         .valid_in2 (intf.valid_in2),
45         .valid_in3 (intf.valid_in3),

```

```
46     .data_out0 (intf.data_out0),
47     .data_out1 (intf.data_out1),
48     .valid_out0 (intf.valid_out0),
49     .valid_out1 (intf.valid_out1)
50 );
51
52 // Testbench Registration & Run Test
53 initial begin
54     // Set virtual interface in uvm_config_db for all components
55     uvm_config_db#(virtual router_if)::set(null, "*", "vif", intf);
56
57     // Run the UVM Test
58     run_test("router_test");
59 end
60
61 endmodule : top
```

Conclusion

The UVM verification environment for `router.v` was successfully designed, implemented, and verified. Key achievements include:

- Full modular UVM architecture with dynamic active/passive agent configuration via `uvm_config_db`.
- Complete UVM verbosity filtering demonstration across 4 simulation passes with exact message count validation.
- **100.00% Functional Coverage** achieved across all valid signals, byte boundary values, and priority cross-coverage.
- 100% scoreboard pass rate (236 PASS, 0 FAIL) with zero errors and zero warnings.

All deliverables, including SystemVerilog code files, automated `run.do` script, PDF report, and submission archives (`.zip` and `.rar`) have been compiled and organized.